05. Python Fundamentals (Part-5)

Concepts: File I/O, Exception Handling, List Comprehensions, JSON Module

Author: Golam Maula Lincoln
GitHub Profile: https://github.com/gmlincoln

Visit GitHub Profile

1. File I/O (Input/Output)

File I/O means reading data from files and writing data to files using Python. Python provides built-in functions to work with files.

Opening a File

We use the `open()` function to open a file. It takes two primary arguments: filename and mode.

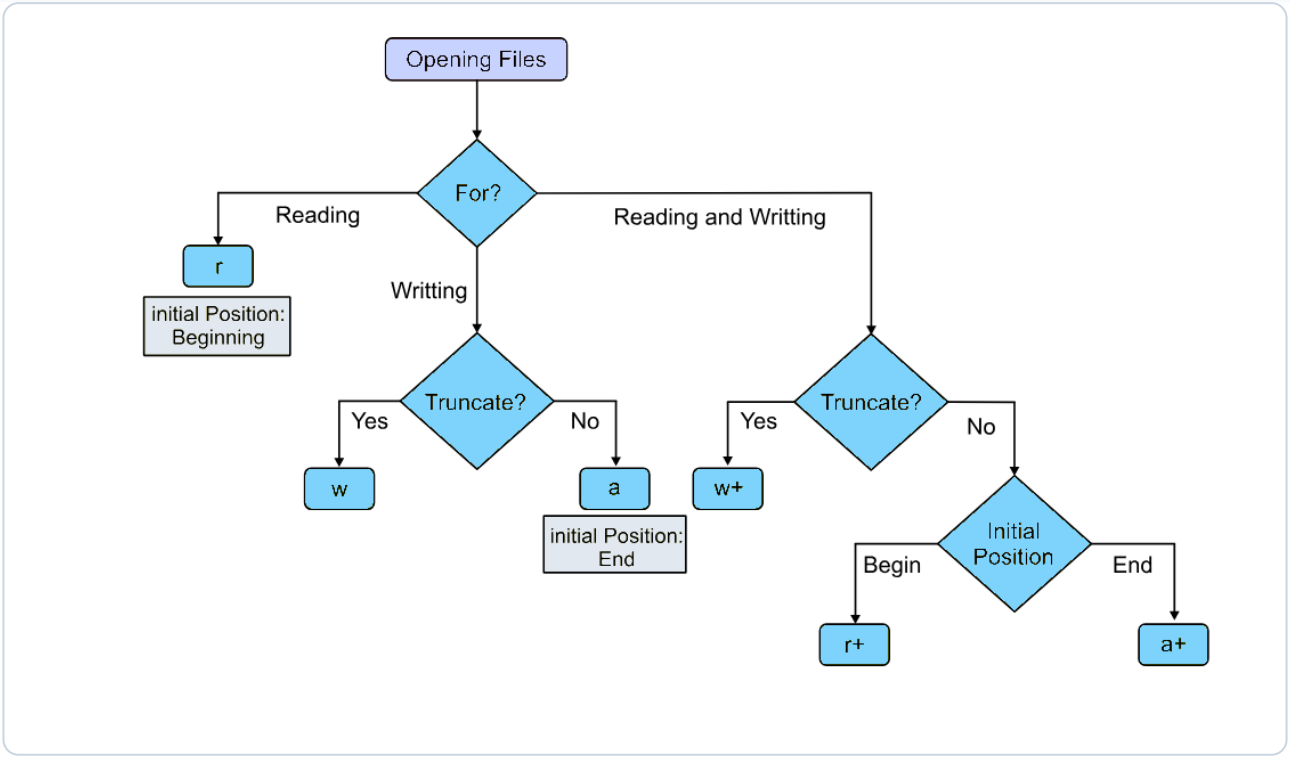
```
file = open("filename", "mode")
```

Common Modes

Mode	Meaning	Description
r	Read	Opens file for reading (default). Error if file doesn't exist.
w	Write	Creates new file or overwrites existing file.
a	Append	Adds content at the end of the file.
r+	Read + Write	File must exist. Allows reading and writing.
w+	Write + Read	Overwrites existing file. Allows writing and reading.
a+	Append + Read	Reads + adds to end of file.
b	Binary Mode	For images, videos, etc. (use with other modes like `rb`).

What to use when?

Follow this flowchart to choose the right file mode for your scenario:



Reading from a File

We have multiple functions to read content from a file:

- `read()` - Reads entire file as a single string.
- `readline()` - Reads one line at a time.
- `readlines()` - Reads all lines into a list.

```
# Using read()
f = open("data.txt", "r")
content = f.read()
print(content)
f.close()

# Using readline() and readlines()
f = open("data.txt", "r")
line1 = f.readline()
lines = f.readlines()
f.close()
```

Writing to a File

- `write()` - Writes a string to a file.
- `writelines()` - Writes multiple lines at once.

```
# Using write()
f = open("data.txt", "w")
f.write("Hello students!")
f.close()

# Using writelines()
f = open("data.txt", "a")
f.writelines(["Line 1
", "Line 2
"])
f.close()
```

Recommended Usage & Deleting a File

Always remember to close a file to free system resources. We use a context manager (`with` block) that automatically closes the file.

```
with open("data.txt", "r") as f:
    content = f.read()
    print(content)

# Deleting a File
import os
os.remove("data.txt")
```

2. Exception Handling

An exception is an error that occurs while a program is running. If not handled, the program crashes. Exception Handling allows you to manage errors gracefully so your program continues to run.

Examples of Exceptions

- Dividing by zero → `ZeroDivisionError`
- Using an undefined variable → `NameError`
- Opening a missing file → `FileNotFoundError`
- Wrong data type → `TypeError`

Basic Syntax

```
try:
    # Code that may cause an error
    x = 10 / 0
except:
    # Code that runs if error occurs
    print("Error occurred!")
```

We can also throw specific exceptions, and use `else` and `finally` blocks.

- **`else` Block:** Executes only if **no exception** happens.
- **`finally` Block: Always executes**, whether an exception occurs or not. Used for cleanup tasks (closing files, releasing resources).

```
try:
    f = open("data.txt")
    print(f.read())
except FileNotFoundError:
    print("File not found!")
else:
    print("Read successful!")
finally:
    print("Execution completed.")
```

3. List Comprehensions

List Comprehension is a short and elegant way to create lists in Python. It replaces long `for` loops with one-line expressions.

Basic Syntax

```
[expression for item in iterable]

# Create a list of numbers 1 to 5
nums = [x for x in range(1, 6)]
print(nums)  # Output: [1, 2, 3, 4, 5]
```

List Comprehension with Condition

```
[expression for item in iterable if condition]

# Even numbers from 1 to 10
evens = [x for x in range(1, 11) if x % 2 == 0]
print(evens)  # Output: [2, 4, 6, 8, 10]
```

List Comprehension with if-else

```
[expression_if_true if condition else expression_if_false for item in iterable]

# Label numbers as "Even" or "Odd"
labels = ["Even" if x % 2 == 0 else "Odd" for x in range(1, 6)]
print(labels)  # Output: ['Odd', 'Even', 'Odd', 'Even', 'Odd']
```

4. Working with JSON Module

JSON (JavaScript Object Notation) is a lightweight data format used to exchange data between programs, APIs, websites, etc. The JSON format is very similar to Python dictionaries.

Python's `json` module allows you to read, write, encode, and decode JSON.

```
import json
```

Converting Python to JSON & JSON to Python

- `json.dumps()` - Converts Python objects into a JSON string.
- `json.loads()` - Converts a JSON string to a Python dictionary.

```python
data = {
    "name": "John",
    "age": 25,
    "marks": [85, 90, 92]
}

# Python to JSON
json_string = json.dumps(data)
print(json_string)

# JSON to Python
json_data = '{name": "John", "age": 25}'
python_obj = json.loads(json_data)
print(python_obj["name"])  # Output: John
```

Reading & Writing JSON Files

- `json.load(f)` - Reads JSON from a file.
- `json.dump(data, f)` - Writes JSON to a file. (Use `indent=4` to make the JSON neat and readable).

```python
# Writing JSON to a file
data2 = {"name": "Aisha", "city": "Delhi"}
with open("data.json", "w") as f:
    json.dump(data2, f, indent=4)

# Reading JSON from a file
with open("data.json", "r") as f:
    loaded_data = json.load(f)
    print(loaded_data)
```

✨ Keep Learning & Keep Exploring! ✨
Document created for GitHub profile: **gmlincoln (Golam Maula Lincoln)**